

double underscores between `author` (a field on the `Article` model), and `first_name` (a field in the `user` model). In object oriented parlance this is equivalent to `article.author.first_name.startswith('v')`.

There are a lot many more ways we can filter and look for the content we need. Like the `'startswith'` trick we used above, you can also use `creation_date_gt` to check for publish dates greater (hence `gt`) than the supplied date, or `_lt` for less than. For example filtering with `'creation_date_lt='2014-02-14'` will return all articles that were created before February 14th 2014.

You can also supply multiple filters at once, or chain filters. So the following both return the same result, articles with a title that starts with `'A'` that are not published:

- `Article.objects.filter(title__startswith='A', publish_status=False)`
- `Article.objects.filter(title__startswith='A').filter(publish_status=False)`

The above are both equally efficient. You can even chain an `exclude` after a filter.

You can also sort / order the results based on one of the fields using the `'order_by'` method. For example:

- `Article.objects.order_by('title')`

The above will order the articles by their title. Of course you can follow this up with additional filter calls or use the `'order_by'` method after filter calls.

You will also be pleased to know that the result of all of these operations behaves like a Python list (although it isn't one). So you can use the familiar list operations such as accessing a particular index with `'[index]'` or a range as `'[start:end]'` or even a strided range with `'[start:end:stride]'`.

Modifying Data

Now that we have a bunch of data in our database, we should can go about modifying it and saving it back to the db. This is as simple as querying the db for a single object, editing its properties and using the `'save'` method just as we did while creating a new entry!

If you know the id of any of your articles, you can fetch it using the `'get'` method as follows:

- `article2 = Article.objects.get(pk=11)`

The above will fetch the article with an `pk` (primary key aka id) of 11. Now you can modify the title, contents or any other field just like before,